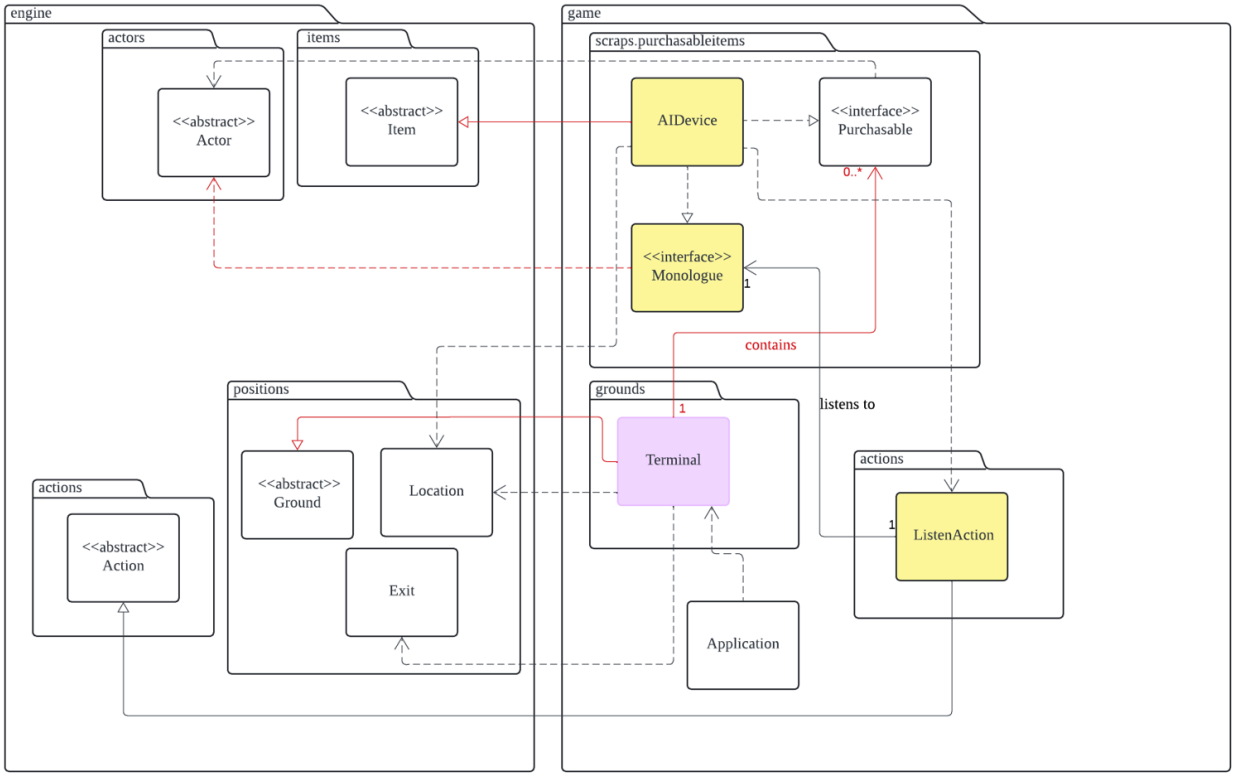
Assignment 3 Design Rationale
Requirement 3: dQw4w9WgXcQ

UML Diagram:



Design Goal:
The design goal for this assignment is to create an AI device named Astley which can be bought from the terminal for 50 credits. The AI device has a subscription fee to pay to listen to its monologue. Every five ticks, the intern must pay 1 credit as subscription fee. If intern cannot pay for the subscription fee, the intern cannot interact with the AI device. The intern can choose to listen to the monologue of this AI device when it is still on subscription. There will be default monologues that the AI device has but also some other monologues which are only available under some certain conditions.

Design Decision:
I created a new class called AIDevice that extends Items. I also created an interface called Monologue so that for every actor/item that can have monologue, they can implement the interface. In this interface, it has a getMonologue() abstract method. This method would get the arraylist of possible monologue that the actor/item/ground could have. This would allow for extensibility since any elements that have monologue can just implement this surface. Then I make AIDevice to implement Purchasable and Monologue interface. To make the Intern to pay for subscription fee when having this

AIDevice in the inventory, I override the tick(Location currentLocation, Actor actor) method and implement the logic there. This method specifically only gets called when an item is inside the intern's inventory since subscription should be paused according to the requirement. I also override the getMonologue() method to return an arraylist of monologues. Besides, I created a ListenAction class that extends Action class, this ListenAction instance would be created and can be performed by intern in the allowableActions() in AIDevice class if the AIDevice is still in the subscription period. The ListenAction has a target which is of type Monologue, and it would just get the arraylist consist of possible monologues of that target and randomly return one monologue when executed. This allows for scalability since as long as any other actor/item that has monologue could be used as a target for ListenAction without having different attributes with different types to store different target.

Alternative Design:
One alternative approach is to have a AIDevice class that stores default monologue strings and ListenAction has one attribute of type AIDevice that store AIDevice instance. But in ListenAction class's execute() method can contain all the logics for getting all the possible monologue of AIDevice. And if there is another item/actor that has monologue, the execute() method also contain all the logics for getting the monologue instead of having the logic to get monologue in each respective class. In other words, all the logic is implemented in ListenAction class and other classes are just empty class. However, this would violate **Single Responsibility Principle (SRP)**, since ListenAction is acting like a GOD class that handles too much and the other classes (in this case, AIDevice) would just be data classes, which is one of the code smells. Another code smell would be large classes due to how ListenAction is having many responsibilities. It would have connascence of name, but the problem is the connascence has low locality since now the coupled elements are now in different classes which is worse than coupled elements within same class.

Besides, another alternative could be having different attributes of different types in ListenAction to store the different instances that can have monologue (e.g. attribute of type AIDevice store AIDevice instance, attribute of type Crab store crab instance if in future, there is a crab can talk). And in the execute() method, it would check which attributes is not null then perform getting the monologue logics there which involves a lot of if statement. When adding new item/actor/ground that can speak, we would need to modify ListenAction class which violates **Open-Closed Principle (OCP)**. It also involves high strength of connascence of execution since we need to check if the attribute that store the instance is not null before we can get the monologue to make sure that we are actually listening to this instance instead of other instance of different type. This also involves a procedural programming code smell which is a sequence of if statements. Not

only that, this approach of having different attributes of different types in ListenAction to store the different instances that can have monologue would also violate **Dependency Inversion Principle (DIP)**. This is because now the ListenAction class is depending on concrete classes (such as depending on AIDevice) instead of abstraction. It would also have high connascence of type since ListenAction directly depends on each different concrete classes that has monologue.

Analysis of Alternative Design:
The alternative design is not ideal because it violates various Design and SOLID principles and have design smells:

1. **Single Responsibility Principle (SRP):**
    - It states that one class should have only one responsibility instead of multiple responsibilities
    - ListenAction execute() method contains all the logic for getting the monologue instead of having the logic implemented in each respective classes, so ListenAction has too many responsibilities
2. **Open-Closed Principle (OCP):**
    - each software entity should be only open for extension but closed for modification.
    - having different attributes of different types in ListenAction to store the different instances that can have monologue would involves a lot of if statements to check which is not null and adding new item/actor/ground that has monologue would need to modify existing class.
    - We need to add new attribute of that new type and add new if statement and add new logic for getting the monologue of that attribute.
3. **Dependency Inversion Principle (DIP):**
    - States that a concrete class should not depend on another concrete class, instead, it should depend on abstractions.
    - Having different attributes of different types in ListenAction to store the different instances would mean having strong dependency with the concrete classes which violate **DIP** since ListenAction don't depends on abstraction.
4. **Procedural programming smell: Data classes**
    - Since all the logic for getting the monologue is done in ListenAction class, other classes such as AIDevice would not have any logic in it, it just stores the default monologue strings and does not operate on the data it owns.
5. **Code length smell: Large classes**
    - Since ListenAction is acting like a GOD class, it would also have this design smell which basically indicates it is having too much responsibilities.
6. **Connascence of Execution**

- Since now ListenAction contains a lot of attributes that store different type of instance, it needs to check which is not null first so that it knows what is the target to listen to, therefore, it is important that we must check for whether the attribute is not null first before getting the monologue.

7. **Procedural programming smell: sequence of if statements**
    - The ListenAction of the alternative design involves checking whether which attribute is not null which involves a lot of if statements
8. **High connascence of type**
    - Because ListenAction directly depends on each different concrete classes that has monologues such as AIDevice.

Final Design:

In our design, we closely adhere to SOLID principles and reduce code smells and reduce strong connascence.

1. Single Responsibility Principle (SRP):

- **Application:** In the current design that we choose, ListenAction does not contain all the logic to handle getting all the monologues, instead, this responsibility is by the AIDevice class. It also does not have different types of attributes that handles getting monologues in different way.
- **Why:** So, it adheres to SRP since it only has one responsibility. This would allow easier maintenance for future.
- **Pros:** Easier to maintain and scale. We eliminated design smell such as data classes since now there is logic for getting monologue in AIDevice class, compared to where AIDevice is just a data class with no logic in it. Also eliminate large classes code smell since the current design now adheres to SRP.
- **Cons:** Increased in number of classes which might be hard to understand for new developers

2. Open-Closed Principle (OCP):

- **Application:** We make use of an interface (Monologue) which has a getMonologue() method and through this, we just need to have one attribute of type Monologue in ListenAction. And we can call getMonologue() method in execute(). This design adheres to OCP.
- **Why:** If there is a new ground/item/actor that can have monologue, the new class just needs to implement Monologue interface and override getMonologue() method. Therefore, there is no need to modify existing class when adding new functionality.
- **Pros:** Easier to extend. Through our current design, we eliminated code smell like sequence of if statement through interface and polymorphism. We also

reduce connascence of execution down to connascence of name since now there is only one attribute of type Monologue in ListenAction which would get the job done.

- **Cons:** An interface is introduced which might need more time for new developer to familiarize with.

3. **Dependency Inversion Principle (DIP):**
    - **Application:** Through abstraction and creating an interface, now the attribute(targetToListenTo) in ListenAction class is type Monologue and ListenAction now depends on abstraction instead of concrete class
    - **Why:** This would help enhance flexibility in our implementation in the future if we decided to add new actor/item/ground that has monologue. So if low level code changes, it does not mean the high level code needs to be changed.
    - **Pros:** Enhance flexibility. Besides, the current design that we chose now only depends on Monologue interface which reduce high connascence of type to lower connascence of type.
    - **Cons:** An interface is introduced which might need more time for new developer to familiarize with

Conclusion:

Overall, our chosen design provides a framework which achieve the requirements listed without violating SOLID principles, without having design smells, and still achieves maintainability and extensibility.